```
    if (!apiKey) {
      throw new Error('Google API Key is required');
    }
    this.apiKey = apiKey;
    this.embeddings = new GoogleGenerativeAIEmbeddings({ apiKey });
    this.retriever = retriever; // Injected dependency
    this.llm = llm; // Injected dependency
  }
...
}
```

In this refactored version of `RAGSystem`, both the retriever and the LLM are passed as parameters to the constructor. This design pattern enhances testability because, during unit tests, you can easily inject mock instances of `FakeRetriever` and `FakeLLM` to enable controlled testing environments where you can simulate various behaviors and responses.

I encourage you to review your existing code and refactor it to enhance testability without compromising readability or usability. Refactoring for testability not only simplifies the testing process but also ensures that your code remains modular and easy to maintain.

Summary

- When troubleshooting, developers should
  - Check that mock responses match expected formats and content.
  - Verify error handling by simulating various error conditions.
  - Ensure asynchronous operations are properly handled in tests.
  - Validate that streaming behaviors work as intended.
  - Confirm that custom events are properly emitted and handled.
- Effective debugging of AI web applications requires a systematic and investigative approach.
- Mocking external dependencies in AI application tests allows for controlled, reproducible test environments that are important for maintaining code reliability.
- Unit testing individual components of AI systems, such as document processing and text summarization, ensures each part functions correctly in isolation.
- Integration testing of AI chains and workflows verifies that multiple components work together as expected, simulating real-world usage scenarios.
- Robust testing frameworks for AI applications should include checks for error handling, edge cases, and varying AI model outputs to ensure stability.
- Testing LangChain.js-powered applications involves simulating AI model responses and document retrieval using functions like `FakeLLM` and `FakeRetriever` testing helpers.